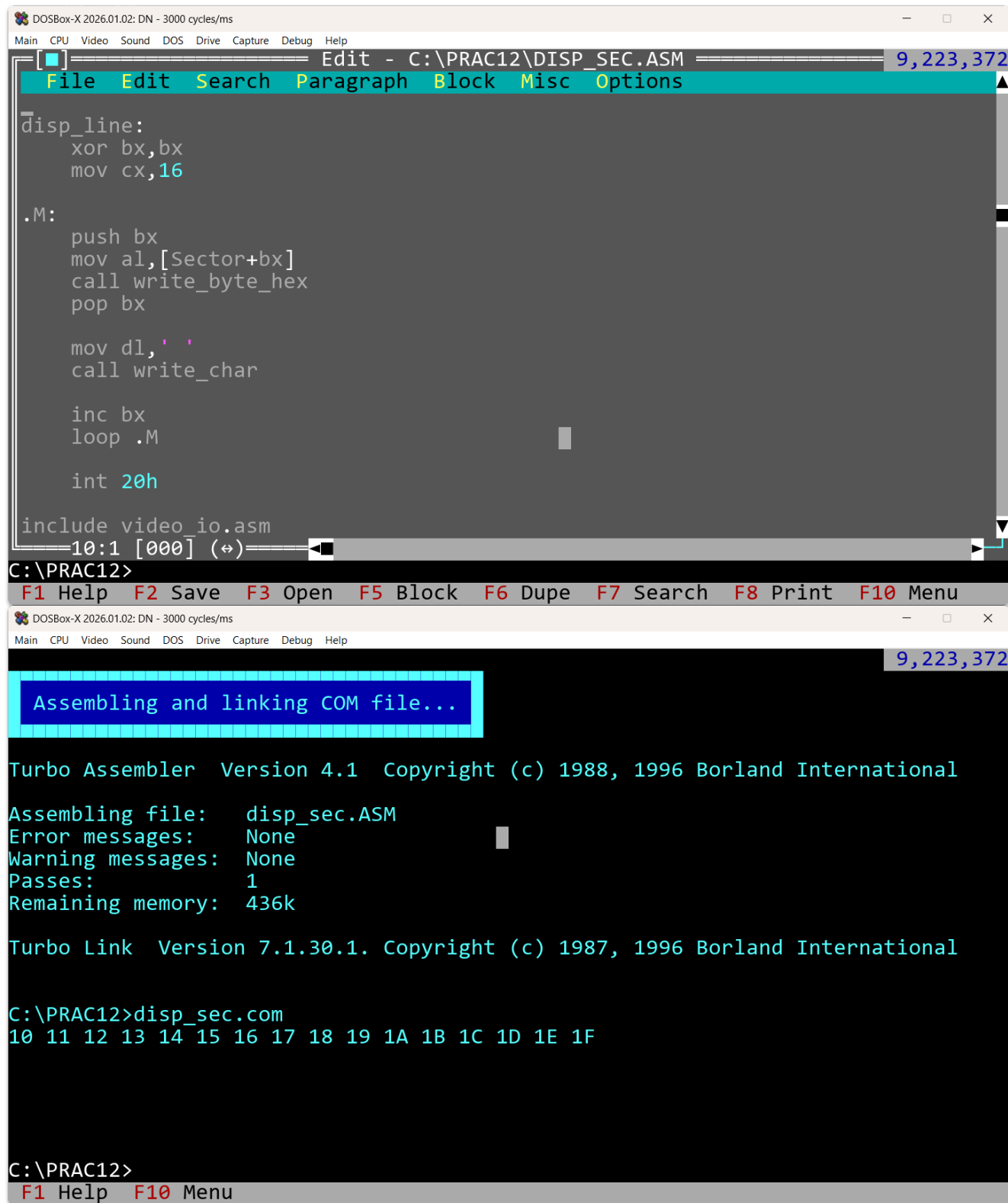


Лабораторный практикум 12. Фролов А.А.

Дампирование памяти.

Дампирование 16 байтов:



The screenshot shows two windows from DOSBox-X. The top window is an editor titled 'Edit - C:\PRAC12\DISP_SEC.ASM' showing assembly code. The bottom window shows the output of Turbo Assembler and Turbo Link, followed by a hex dump of the generated COM file.

```
dos_line:
  xor bx,bx
  mov cx,16

.M:
  push bx
  mov al,[Sector+bx]
  call write_byte_hex
  pop bx

  mov dl,' '
  call write_char

  inc bx
  loop .M

  int 20h

include video_io.asm
```

C:\PRAC12>

F1 Help F2 Save F3 Open F5 Block F6 Dupe F7 Search F8 Print F10 Menu

DOSBox-X 2026.01.02: DN - 3000 cycles/ms

Assembling and linking COM file...

Turbo Assembler Version 4.1 Copyright (c) 1988, 1996 Borland International

Assembling file: disp_sec.ASM
Error messages: None
Warning messages: None
Passes: 1
Remaining memory: 436k

Turbo Link Version 7.1.30.1. Copyright (c) 1987, 1996 Borland International

C:\PRAC12>disp_sec.com

10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F

C:\PRAC12>

F1 Help F10 Menu

Дампирование 256 байт:

Исправленный файл disp_sec.asm

```

1  .model tiny
2  .code
3  org 100h
4
5  start:
6      jmp disp_sector
7
8  ; Данные программы: 256 байтов
9  Sector db 16 dup(10h)
10         db 16 dup(11h)
11         db 16 dup(12h)
12         db 16 dup(13h)
13         db 16 dup(14h)
14         db 16 dup(15h)
15         db 16 dup(16h)
16         db 16 dup(17h)
17         db 16 dup(18h)
18         db 16 dup(19h)
19         db 16 dup(1Ah)
20         db 16 dup(1Bh)
21         db 16 dup(1Ch)
22         db 16 dup(1Dh)
23         db 16 dup(1Eh)
24         db 16 dup(1Fh)
25
26 ; -----
27 ; Вывод сектора: 256 байтов = 16 строк по 16 байтов
28 ; -----
29 disp_sector:
30     xor dx,dx          ; DX = номер первого байта строки
31     mov cx,16          ; всего 16 строк
32
33 sector_loop:
34     call disp_line      ; вывести одну строку
35     call Send_crlf      ; перейти на новую строку
36
37     add dx,16           ; следующая строка начинается через 16 байтов
38     loop sector_loop
39
40     int 20h            ; выход в DOS
41
42 ; -----
43 ; Вывод одной строки: 16 байтов
44 ; Вход: DX = номер первого байта строки в Sector
45 ; -----
46 disp_line:
47     push bx
48     push cx
49     push dx

```

```

50
51     mov bx,dx          ; BX = смещение внутри Sector
52     mov cx,16          ; 16 байтов в строке
53
54 line_loop:
55     push bx
56     mov al,[Sector+bx]
57     call write_byte_hex
58     pop bx
59
60     mov dl,' '
61     call write_char
62
63     inc bx
64     loop line_loop
65
66     pop dx
67     pop cx
68     pop bx
69     ret
70
71 include video_io.asm
72 include cursor.asm
73
74 end start

```

cursor.asm:

```

1  .model tiny
2  .code
3
4  cr equ 13
5  lf equ 10
6
7  send_crlf:
8      push ax
9      push dx
10     mov ah,2
11     mov dl,cr
12     int 21h
13     mov dl,lf
14     int 21h
15     pop dx
16     pop ax
17     ret

```

Результат:

```
DOSBox-X 2026.01.02: DN - 3000 cycles/ms
Main CPU Video Sound DOS Drive Capture Debug Help
Passes: 1
Remaining memory: 434k

Turbo Link Version 7.1.30.1. Copyright (c) 1987, 1996 Borland Intern

C:\PRAC12>disp_sec.com
10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10
11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11
12 12 12 12 12 12 12 12 12 12 12 12 12 12 12 12
13 13 13 13 13 13 13 13 13 13 13 13 13 13 13 13
14 14 14 14 14 14 14 14 14 14 14 14 14 14 14 14
15 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15
16 16 16 16 16 16 16 16 16 16 16 16 16 16 16 16
17 17 17 17 17 17 17 17 17 17 17 17 17 17 17 17
18 18 18 18 18 18 18 18 18 18 18 18 18 18 18 18
19 19 19 19 19 19 19 19 19 19 19 19 19 19 19 19
1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A
1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B
1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1
1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1
1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1
1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1
C:\PRAC12>
F1 Help F10 Menu
```

Очистка экрана:

```
1 cr equ 13
2 lf equ 10
3
4 ; -----
5 ; Send_crLf
6 ; Переход на новую строку
7 ; -----
8 Send_crLf proc
9     push ax
10    push dx
11
12    mov ah,2
13    mov dl,cr
14    int 21h
15
16    mov dl,lf
17    int 21h
18
19    pop dx
20    pop ax
21    ret
22 Send_crLf endp
23
24
```

```

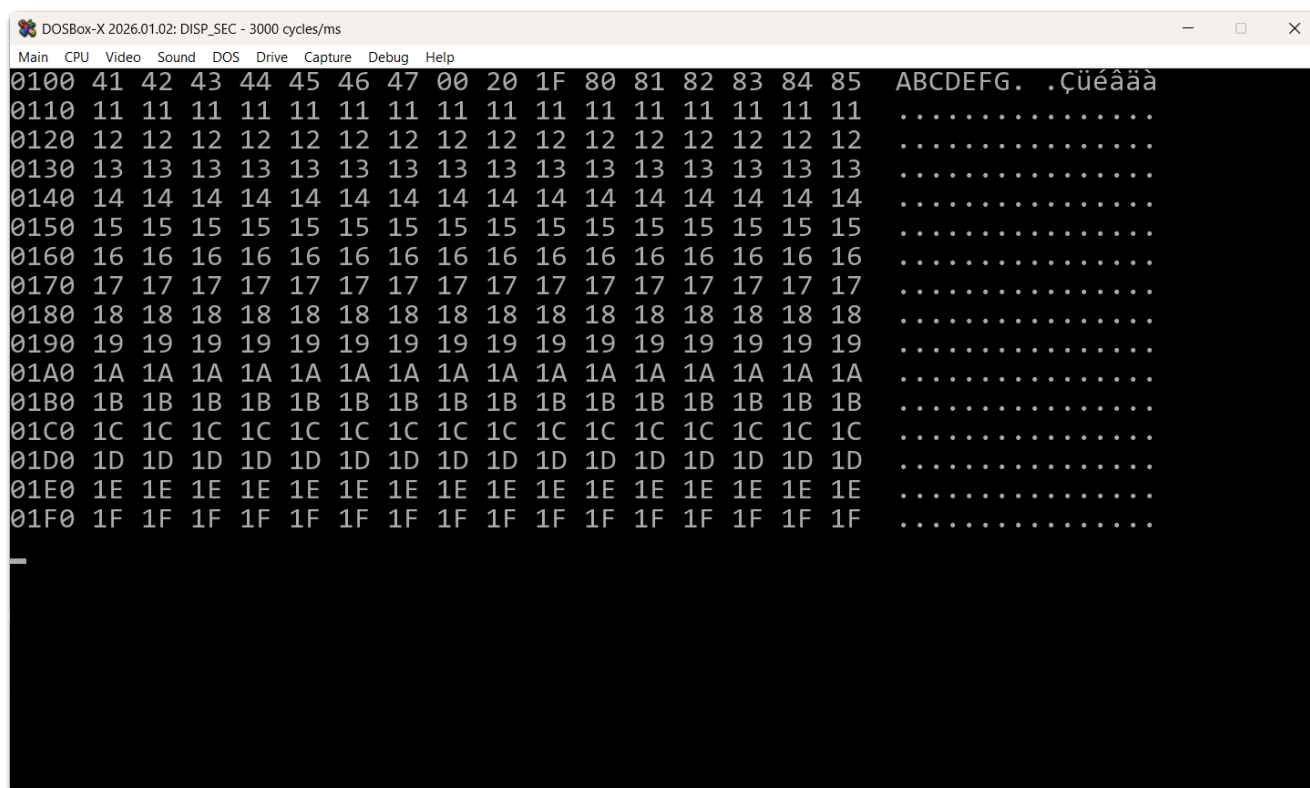
25 ; -----
26 ; Clear_screen
27 ; Очистка экрана через BIOS int 10h
28 ; -----
29 Clear_screen proc
30     push ax
31     push bx
32     push cx
33     push dx
34
35     xor al,al
36     xor cx,cx
37     mov dh,24
38     mov dl,79
39     mov bh,7
40     mov ah,6
41     int 10h
42
43     pop dx
44     pop cx
45     pop bx
46     pop ax
47     ret
48 Clear_screen endp

```

С очисткой экрана:

“Лабораторный практикум 12. Фролов А.А.-3.png” не может быть найдена.

Лабораторная работа



В ходе лабораторной работы была доработана программа дампирования памяти. Изначально программа выводила 256 байтов памяти в шестнадцатеричном виде по 16 байтов на строку. После доработки каждая строка дампа стала содержать адрес первого байта строки, 16 байтов в HEX-виде и символьное представление этих же байтов.

Для хранения начального адреса области памяти была добавлена переменная:

```
1 Address dw 100h
```

Она задаёт начальный адрес области `Sector`. Адрес каждой строки рассчитывается как сумма `Address` и смещения текущей строки.

В процедуру вывода сектора была добавлена очистка экрана перед выводом дампа:

```
1 disp_sector:
2     call Clear_screen
3
4     xor dx,dx
5     mov cx,16
```

Также в конце программы было добавлено ожидание нажатия клавиши, чтобы результат не исчезал сразу после выполнения:

```
1     mov ah,0
2     int 16h
3
4     int 20h
```

Основные изменения были внесены в процедуру `disp_line`. Теперь она сначала выводит адрес строки:

```
1     mov bx,dx
2
3     push dx
4     add dx,[Address]
5     call write_word_hex
6     pop dx
7
8     mov dl,' '
9     call write_char
```

После этого выводятся 16 байтов в шестнадцатеричном виде:

```
1     push bx
2     mov cx,16
```

```

3
4     hex_loop:
5         mov al,[Sector+bx]
6         call write_byte_hex
7
8         mov dl,' '
9         call write_char
10
11        inc bx
12        loop hex_loop

```

Затем программа возвращается к началу этой же строки и выводит байты как символы:

```

1     mov dl,' '
2     call write_char
3
4     pop bx
5     mov cx,16
6
7     char_loop:
8         mov dl,[Sector+bx]
9         call write_char
10
11        inc bx
12        loop char_loop

```

Для корректного отображения управляющих символов была изменена процедура `write_char`. Если код символа находится в диапазоне от `00h` до `1Fh`, вместо него выводится точка:

```

1     write_char proc
2         cmp dl,1Fh
3         ja print_char
4
5         mov dl,'.'
6
7     print_char:
8         mov ah,02h
9         int 21h
10        ret
11     write_char endp

```

Также была добавлена процедура очистки экрана через BIOS:

```

1     Clear_screen proc
2         push ax
3         push bx

```

```
4      push cx
5      push dx
6
7      xor al,al
8      xor cx,cx
9      mov dh,24
10     mov dl,79
11     mov bh,7
12     mov ah,6
13     int 10h
14
15     mov ah,2
16     mov bh,0
17     mov dh,0
18     mov dl,0
19     int 10h
20
21     pop dx
22     pop cx
23     pop bx
24     pop ax
25     ret
26 Clear_screen endp
```

В результате программа выводит дамп памяти в удобном виде: слева отображается адрес строки, далее идут 16 байтов в шестнадцатеричном формате, а справа выводится символьное представление этих байтов. Управляющие символы заменяются точками, поэтому вывод не ломает структуру дампа.